Arrays and Strings

PART 1 : ARRAYS **3**
1. Introduction to Arrays **3**
Initialization of Array **4**
Declaration with Initialization **5**
Declaration with Size Omitted **6**
Partial Initialization **6**
Example 7
Memory Representation 7
Complete Zero Initialization **7**
Character Array Declaration **8**
Float Array Declaration 8
Double Array Declaration **8**
Long Integer Array **8**
Using Macro for Array Size **9**
Variable Length Array (C99) **9**
Invalid Array Declarations **9**
Accessing Array Elements **10**
Types of Arrays **12**
One Dimensional Array **12**
Different Array Notations 14
Reversing Array Printing **19**
Reverse Array 20
Searching Algorithms **21**
Sorting Algorithms **26**
1.Bubble Sort 26
2.Selection Sort **29**
3. Insertion Sort **34**
4.Quick Sort 39
Two-Dimensional Array **53**
Declaration and Initialization of Two-Dimensional Arrays **54**
1. Declaration Only (Valid) **54**
2. Complete Initialization (Valid) **55**
3. Flat (Row-wise) Initialization (Valid) **55**
4. Partial Initialization (Valid) **56**
5. Single Row Initialization (Valid) **56**
6. Omitting Row Size (Valid) **57**

7. Omitting Both Dimensions (Invalid)....57
8. Omitting Column Size During Initialization (Invalid)....57
9. Omitting Column Size (Invalid)....57
10. Excess Initializers (Invalid)....58
11. More Rows Than Declared (Invalid)....58
12. Fewer Elements Than Size (Valid)....58
13. Zero Initialization (Valid)....58
14. Character 2D Array (String Table) (Valid)....59
15. Mixed Initialization Style (Valid)....59
Passing 2D Array to Function....61
Different Ways to Pass a 2D Array....65
Applications of Arrays in Embedded Systems....67
PART 2: STRINGS....67
Introduction to String....67
Definition....67
Memory Diagram....69
String Initialization Methods....70
Character Array vs String....71
Taking String Input from the user....71
String Output....78
Displaying Character by Character Using Loop....80
String.h Library....82
1) strlen()....82
2) strcpy()....83
3)strcat()....84
4)strcmp()....85
5)strchr()....86
6)strstr()....88
String Functions Using Pointer....89
1. strlen() Using Pointer....89
2. strcpy() Using Pointer....90
3. strcmp() Using Pointer....92
4. strcat() Using Pointer....93
5. strchr() Using Pointer....94
Complete Program of All Functions....95
Interview Questions....99

PART 1 : ARRAYS

1. Introduction to Arrays

Array is a collection of similar datatype elements stored in contiguous memory locations.

Suppose you want to store marks of 100 students.

Without array: int m1,m2,m3,m4,m5....

This becomes difficult.

Using array: int marks[100];

Easy to manage.

Why Arrays are Important

Arrays are used for:

- Storing multiple values
- Buffer management
- Sensor data storage
- DMA buffers
- Image processing
- UART communication buffers
- ADC sample storage

Definition of Array

An array is a collection of same datatype elements stored sequentially in memory.

Syntax of Array

datatype array_name[size];

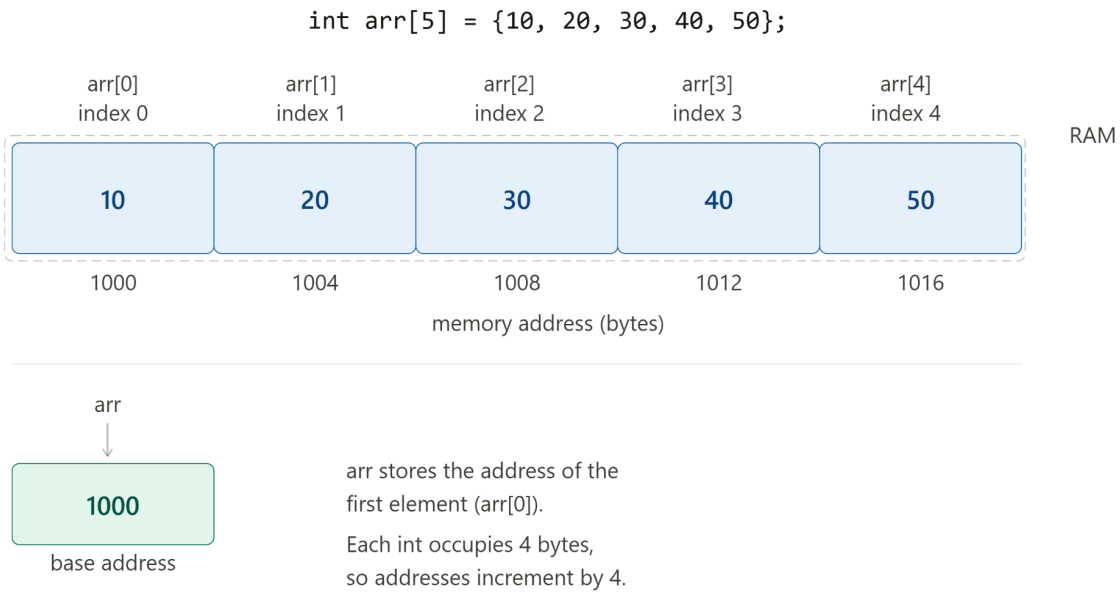
Example: int arr[5];

Memory Representation of Array

Suppose: int arr[5] = {10,20,30,40,50};

Memory:

Address	Value
1000	10
1004	20
1008	30
1012	40
1016	50



Note : Array elements are stored in contiguous memory locations.

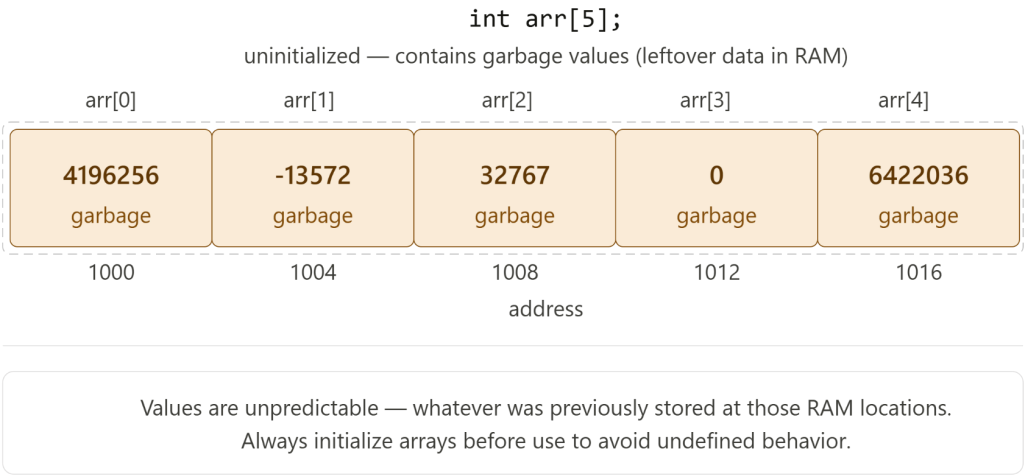
Initialization of Array

Declaration Only

Syntax
data_type array_name[size];

Example : int arr[5];

Memory Representation :



Here:

- Data type = int
- Array name = arr
- Size = 5
- Value in memory location = Grabage value

The array can store **5 integer values**.

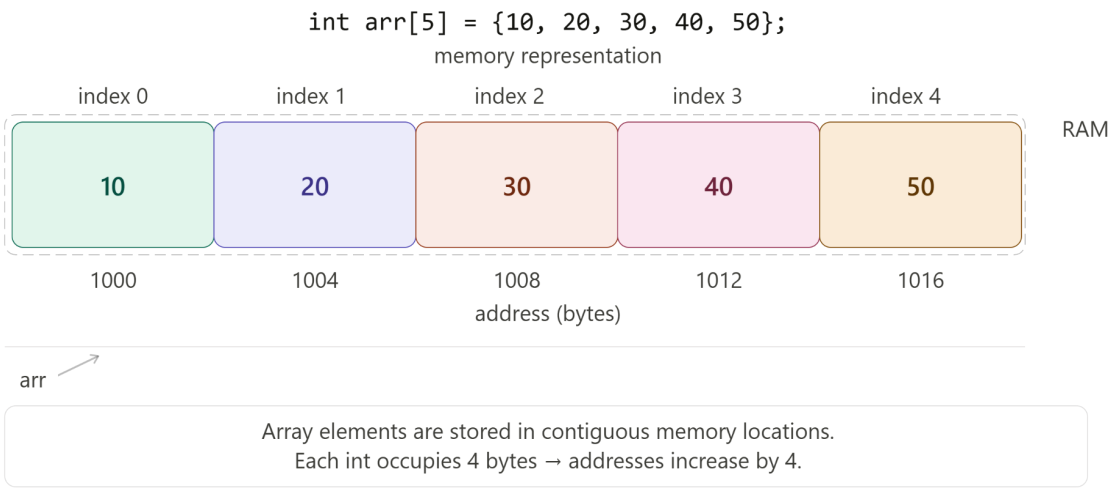
Declaration with Initialization

Syntax
data_type array_name[size] = {values};

Example
int arr[5] = {10,20,30,40,50};

Memory Representation

Index : 0 1 2 3 4
Value :10 20 30 40 50



Declaration with Size Omitted

Compiler automatically counts elements.

Syntax

data_type array_name[] = {values};

Example

int arr[] = {10,20,30,40,50};

Compiler calculates the number of elements is 5.

Equivalent to:

int arr[5] = {10,20,30,40,50};

Partial Initialization

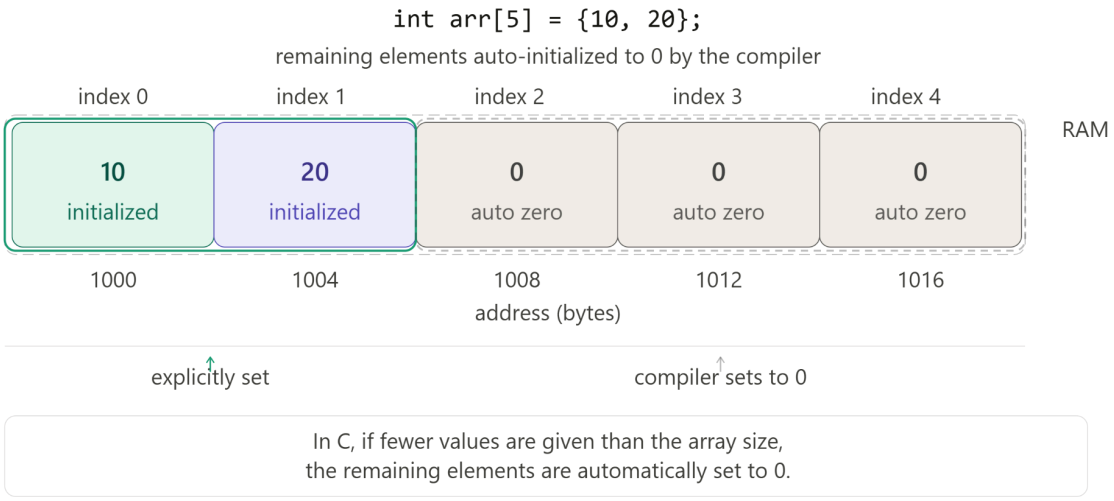
If fewer values are provided, remaining elements become **0**.

Example

int arr[5] = {10,20};

Memory Representation

Index : 0 1 2 3 4
Value :10 20 0 0 0

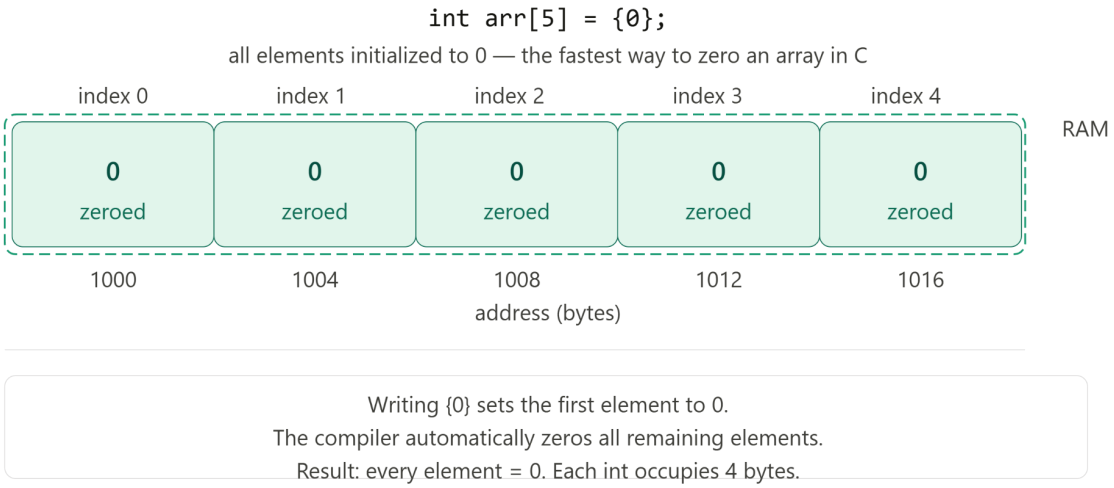


Complete Zero Initialization

Example : int arr[5] = {0};

Memory Representation

Index : 0 1 2 3 4
Value : 0 0 0 0 0



Very commonly used in embedded programming.

Character Array Declaration

Example : char name[5];

Stores characters.

Index : 0 1 2 3 4
Value : ? ? ? ? ?

Float Array Declaration

Example : float marks[5];

Initialization : float marks[5] = {65.5,72.5,80.0,90.5,88.0};

Double Array Declaration

Example : double data[10];

Long Integer Array

Example : long int numbers[100];

Using Macro for Array Size

Example

```
#define SIZE 5
int arr[SIZE];
```

Benefit: If size changes, update only one place.

Variable Length Array (C99)

Size determined at runtime.

Example

```
int n;

scanf("%d",&n);

int arr[n];
```

If user enters: 5

Compiler creates: int arr[5];

Invalid Array Declarations

1) Size Missing without Initialization

Invalid : int arr[];

Compiler doesn't know the size.

2) Negative Size

Invalid : int arr[-5];

3) More Initializers than Size

Invalid : int arr[3]={10,20,30,40};

4 values cannot fit into 3 locations.

Comparison Table

Declaration	Meaning
int arr[5];	Declaration only
int arr[5]={10,20,30,40,50};	Full initialization
int arr[]={10,20,30};	Compiler calculates size
int arr[5]={10,20};	Partial initialization
int arr[5]={0};	All elements become 0
char name[10];	Character array
char str[]="HELLO";	String declaration
float marks[5];	Float array
double data[10];	Double array

Accessing Array Elements

What is Array Element Access?

After creating an array, we need a way to store and retrieve individual values from it.

Each value in an array is called an **element**. We access elements using their **index number (subscript)**.

Syntax

array_name[index]

Example:

arr[0]
arr[1]
arr[2]

where:

- arr → array name
- 0,1,2 → index positions

Note : Array Index Always Starts from 0.

Example Program

```c
#include <stdio.h>   // Header file for printf()

int main()
{
    // Declare and initialize an integer array with 5 elements
    int arr[5] = {10, 20, 30, 40, 50};

    int i;   // Loop counter variable

    // Loop through the array from index 0 to index 4
    for(i = 0; i < 5; i++)
    {
        // Print the current array element
        printf("%d\n", arr[i]);
    }

    // Return 0 to indicate successful program execution
    return 0;
}
```

Array Index Rules

Rule	Description
First index	0
Last index	size-1
Negative index	Invalid
Out of bound access	Undefined behavior

Types of Arrays

1. One Dimensional Array
2. Two Dimensional Array
3. Multi Dimensional Array

One Dimensional Array

A **One-Dimensional Array (1D Array)** is a collection of elements of the same data type stored in **contiguous memory locations** and accessed using a **single index**.

It is called **one-dimensional** because only **one subscript (index)** is required to access an element.

Example: int arr[5];

Taking Array Input from User

```c
#include <stdio.h>
int main()
{
    // Declare an integer array to store 5 elements
    int arr[5];

    // Declare a variable for loop iteration
    int i;

    // Loop to take 5 elements as input from the user
    for(i = 0; i < 5; i++)
    {
        // Read an integer and store it at the current array index
        scanf("%d", &arr[i]);
    }

    // Loop to print all 5 elements of the array
    for(i = 0; i < 5; i++)
    {
        // Print the current array element
        printf("%d ", arr[i]);
```

```
    }

    // Return 0 to indicate successful program execution
    return 0;
}
```

Passing Array to Function

When array is passed to function, base address is passed.

Syntax

void display(int arr[], int size);

OR

void display(int *arr, int size);

Both are the same.

Example Program

```
#include<stdio.h>

// Function declaration
void display(int arr[], int size);

int main()
{
   int arr[5] = {10,20,30,40,50};

  //function call
   display(arr,5);

   return 0;
}

// Function definition
void display(int arr[], int size)
{
   int i;
```

```
    for(i=0;i<size;i++)
    {
        printf("%d ", arr[i]);
    }
}
```

Note : Array name itself acts as pointer to first element.

arr == &arr[0] → base address

Different Array Notations

Array elements can be accessed using different notations because an array name is closely related to pointers.

Consider: int arr[5] = {10,20,30,40,50};

Memory:

Index : 0 1 2 3 4
Value : 10 20 30 40 50

1. Subscript Notation (Most Common)

Syntax
arr[index]

Examples
arr[0]
arr[1]
arr[2]

Program

```
#include <stdio.h>

int main()
{
   int arr[5] = {10,20,30,40,50};
```

```
    printf("%d\n", arr[0]);
    printf("%d\n", arr[3]);

    return 0;
}
```

Output:

10
40

2. Pointer Notation

Since array name stores the base address (**arr)** is equivalent to **&arr[0].** Therefore,

***(arr + index)** can be used.

Examples

*(arr+0)
*(arr+1)
*(arr+2)

Program

```
#include <stdio.h>

int main()
{
    int arr[5]={10,20,30,40,50};

    printf("%d\n", *(arr+0));
    printf("%d\n", *(arr+3));

    return 0;
}
```

Output:

10
40

Relationship Between the Two:

Subscript Notation	Pointer Notation
arr[0]	*(arr+0)
arr[1]	*(arr+1)
arr[2]	*(arr+2)
arr[3]	*(arr+3)
arr[4]	*(arr+4)

Both give exactly the same result.

3. Reverse Subscript Notation (Rare but Valid)

Since: arr[i] → Means: *(arr+i)

and addition is commutative *(i+arr) is also valid.

Therefore: i[arr]

is legal C syntax.

Example

```c
#include <stdio.h>

int main()
{
   int arr[5]={10,20,30,40,50};

   printf("%d\n", 2[arr]);

   return 0;
}
```

Output: 30

Because 2[arr] means *(2+arr)

which is: arr[2]

4.Using Address Operator

Access address of array elements.

Examples

&arr[0]
&arr[1]
&arr[2]

Program:

```
#include <stdio.h>

int main()
{
   int arr[5]={10,20,30,40,50};

   printf("%p\n",&arr[0]);
   printf("%p\n",&arr[1]);

   return 0;
}
```

Output:

1000
1004

(Addresses vary by system.)

5. Array Name as Base Address

The array name itself represents the address of the first element. **arr is** equivalent to &arr[0].

Example:

printf("%p\n", arr);
printf("%p\n", &arr[0]);

Both print the same address.

Important Identities

For: int arr[5];

Identity 1
arr == &arr[0]

Identity 2
arr[i] == *(arr+i)

Identity 3
i[arr] == arr[i]

Identity 4
&arr[i] == arr+i

Example Showing All Notations

```
#include <stdio.h>

int main()
{
   int arr[5]={10,20,30,40,50};

   printf("%d\n", arr[2]);
   printf("%d\n", *(arr+2));
   printf("%d\n", 2[arr]);

   return 0;
}
```

Output:

30
30
30

All three statements access the same element.

Summary Table

Notation	Example	Meaning
Subscript	arr[2]	Third element

Pointer	*(arr+2)	Third element
Reverse Subscript	2[arr]	Third element
Address	&arr[2]	Address of third element
Base Address	arr	Address of first element

Reversing Array Printing

Method 1 : Using Loop

```
#include<stdio.h>

int main()
{
   int arr[5] = {10,20,30,40,50};

   int i;

   for(i=4;i>=0;i--)
   {
      printf("%d ", arr[i]);
   }

   return 0;
}
```

Method 2: Reverse Array Using Function

```
#include<stdio.h>

// Function to reverse array
void reverse(int arr[], int size)
{
   int i;

   for(i=size-1;i>=0;i--)
   {
      printf("%d ", arr[i]);
```

```
    }
}

int main()
{
    int arr[5] = {1,2,3,4,5};

    reverse(arr,5);

    return 0;
}
```

Method 3 : Reverse Array Using Pointer

```
#include<stdio.h>

int main()
{
    int arr[5] = {10,20,30,40,50};

    int *ptr = &arr[4];

    int i;

    for(i=0;i<5;i++)
    {
        printf("%d ", *ptr);

        ptr--;
    }

    return 0;
}
```

Reverse Array

```
#include <stdio.h>
```

```c
int main()
{
    int n, i, temp;

    // Take the size of the array from the user
    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    // Take array elements from the user
    printf("Enter %d elements:\n", n);

    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Reverse the array by swapping elements
    for(i = 0; i < n / 2; i++)
    {
        temp = arr[i];
        arr[i] = arr[n - 1 - i];
        arr[n - 1 - i] = temp;
    }

    // Print the reversed array
    printf("Reversed array:\n");

    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Searching Algorithms

Searching means finding element in array.

Main searching algorithms:

1. Linear Search
2. Binary Search

Linear Search

Checks elements one by one.

Algorithm

Step 1: Start

Step 2: Read array A, size n, and element key to be searched.

Step 3: Set i = 0

Step 4: Compare A[i] with key

- If A[i] == key, display **"Element Found"** and position (i+1), then go to Step 7.
- Otherwise, increment i.

Step 5: Check whether i < n

- If Yes, go to Step 4.
- If No, display **"Element Not Found"**.

Step 6: Stop

Step 7: Stop

Example Program

```c
#include <stdio.h>
int main()
{
    int n, search, i;
    int found = 0;

    // Take the number of elements from the user
    printf("Enter the number of elements: ");
    scanf("%d", &n);
```

```
    int arr[n];

    // Take array elements from the user
    printf("Enter %d elements:\n", n);

    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Take the element to search
    printf("Enter the element to search: ");
    scanf("%d", &search);

    // Search for the element in the array
    for(i = 0; i < n; i++)
    {
        if(arr[i] == search)
        {
            printf("Element Found\n");

            // Set found to 1 when element is found
            found = 1;

            break;
        }
    }

    // Check if the element was not found
    if(found == 0)
    {
        printf("Element Not Found\n");
    }

    return 0;
}
```

Time Complexity : O(n)

Binary Search

Works only on sorted array.

Algorithm

Step 1: Start

Step 2: Read sorted array A, size n, and search element key.

Step 3: Set

low = 0

high = n - 1

Step 4: Repeat while low <= high

Calculate:

mid = (low + high) / 2

Step 5: Compare A[mid] with key

- If A[mid] == key
  - Display **"Element Found"**
  - Go to Step 8
- If key < A[mid]
  - Set high = mid - 1
- If key > A[mid]
  - Set low = mid + 1

Step 6: Repeat Step 4

Step 7: Display **"Element Not Found"**

Step 8: Stop

Example Program

```
#include <stdio.h>

int main()
{
    int n, search;
```

```
    int low, high, mid;
    int found = 0, i;

    // Take the number of elements from the user
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    // Take array elements from the user
    printf("Enter %d elements in sorted order:\n", n);

    for( i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Take the element to search
    printf("Enter the element to search: ");
    scanf("%d", &search);

    // Initialize low and high
    low = 0;
    high = n - 1;

    // Binary search
    while(low <= high)
    {
        // Calculate the middle index
        mid = (low + high) / 2;

        // Check if the middle element is the search element
        if(arr[mid] == search)
        {
            printf("Element Found\n");
            found = 1;
            break;
        }
        // Search in the right half
        else if(search > arr[mid])
        {
            low = mid + 1;
```

```
        }
        // Search in the left half
        else
        {
            high = mid - 1;
        }
    }

    // If element was not found
    if(found == 0)
    {
        printf("Element Not Found\n");
    }

    return 0;
}
```

Time Complexity : O(log n)

Sorting Algorithms

Sorting means arranging data.

Types:

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

1.Bubble Sort

Largest element moves upward like bubble.

Step 1: Start

Step 2: Read

- Read the number of elements n.

- Read the array elements arr[ ].

Step 3: Perform Bubble Sort

1. Set i = 0.
2. Repeat while i < n - 1:
    - Set j = 0.
    - Repeat while j < n - i - 1:
        - Compare arr[j] and arr[j+1].
        - If arr[j] > arr[j+1], swap them.
        - Increment j.
    - Increment i.

Step 4: Display

- Print the sorted array.

Step 5: Stop

Pseudocode

```
START

Read n
Read arr[0] to arr[n-1]

For i = 0 to n-2
{
   For j = 0 to n-i-2
   {
      If arr[j] > arr[j+1]
      {
         Swap arr[j] and arr[j+1]
      }
   }
}

Print sorted array

STOP
```

Example Program

```c
#include <stdio.h>

int main()
{
    int n;
    int i, j, temp;

    // Take the number of elements from the user
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    // Take array elements from the user
    printf("Enter %d elements:\n", n);

    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Bubble Sort
    // Compare adjacent elements and swap if they are in the wrong order
    for(i = 0; i < n - 1; i++)
    {
        for(j = 0; j < n - 1 - i; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                // Swap the elements
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    // Print the sorted array
    printf("Sorted array:\n");

    for(i = 0; i < n; i++)
```

```
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Bubble Sort Time Complexity : $O(n^2)$

2.Selection Sort

Selection Sort is a sorting algorithm that repeatedly selects the smallest element from the unsorted part of the array and places it at the correct position.

Working Principle

The array is divided into two parts:

Part	Description
Sorted Part	Left side
Unsorted Part	Right side

In every pass:

1. Find minimum element from unsorted array
2. Swap it with current position
3. Increase sorted portion

Algorithm: Selection Sort

Step 1: Start

Step 2: Read

- Read the number of elements n.
- Read the array elements arr[ ].

Step 3: Perform Selection Sort

1. Set i = 0.
2. Repeat while i < n - 1:
    - Assume the current element is the minimum:
        - min = i
    - Set j = i + 1.
    - Repeat while j < n:
        - If arr[j] < arr[min], then set min = j.
        - Increment j.
    - If min ≠ i, swap arr[i] and arr[min].
    - Increment i.

Step 4: Display

- Print the sorted array.

Step 5: Stop

Pseudocode: Selection Sort

```
START

Read n
Read arr[0] to arr[n-1]

For i = 0 to n-2
{
  min = i

  For j = i+1 to n-1
  {
     If arr[j] < arr[min]
     {
        min = j
     }
  }

  If min != i
```

```
  {
     Swap arr[i] and arr[min]
  }
}

Print sorted array

STOP
```

Example

Initial array: 50 20 10 40 30

Pass 1:

Minimum element: 10

Swap with first element: 10 20 50 40 30

Pass 2

Unsorted part: 20 50 40 30

Minimum element: 20

Already in correct position.

Array: 10 20 50 40 30

Pass 3

Minimum element: 30

Swap with 50: 10 20 30 40 50

Array sorted.

Program

```
#include<stdio.h>

int main()
{
```

```
    int arr[5] = {50,20,10,40,30};

    int i,j,min_index,temp;

    // Number of passes
    for(i=0;i<4;i++)
    {
        // Assume current element is minimum
        min_index = i;

        // Find minimum element
        for(j=i+1;j<5;j++)
        {
            if(arr[j] < arr[min_index])
            {
                min_index = j;
            }
        }

        // Swap elements
        temp = arr[i];
        arr[i] = arr[min_index];
        arr[min_index] = temp;
    }

    // Print sorted array
    printf("Sorted Array:\n");

    for(i=0;i<5;i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

Sorted Array: 10 20 30 40 50

Time Complexity

Case	Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Embedded System Applications

Used in:

- Small sensor datasets
- Priority arrangement
- Lookup table sorting
- Memory-constrained systems

Bubble Sort vs Selection Sort

Bubble Sort	Selection Sort
Multiple swaps	One swap per pass
Adjacent comparison	Minimum selection
More data movement	Less data movement

3. Insertion Sort

Insertion Sort is a sorting algorithm that places each element at its correct position in the sorted part of the array.

It works similarly to arranging playing cards in hand.

Working Principle

The array is divided into two parts:

Part	Description
Sorted Part	Left side
Unsorted Part	Right side

In every pass:

1. Pick one element from unsorted part
2. Compare with sorted elements
3. Insert at correct position

Algorithm of Insertion Sort

Step 1: Start

Step 2: Read

- Read the number of elements n.
- Read the array elements arr[].

Step 3: Perform Insertion Sort

1. Set i = 1.
2. Repeat while i < n:
    - Store the current element in key = arr[i].

- Set j = i - 1.
- While j >= 0 and arr[j] > key:
  - Move arr[j] one position to the right:
    - arr[j+1] = arr[j]
  - Decrement j.
- Insert key at its correct position:
  - arr[j+1] = key
- Increment i.

Step 4: Display

- Print the sorted array.

Step 5: Stop

Pseudocode: Insertion Sort

```
START

Read n
Read arr[0] to arr[n-1]

For i = 1 to n-1
{
  key = arr[i]
  j = i - 1

  While (j >= 0 AND arr[j] > key)
  {
     arr[j+1] = arr[j]
     j = j - 1
  }

  arr[j+1] = key
}

Print sorted array

STOP
```

Example

Initial array: 50 20 10 40 30

Pass 1

Key element: 20

Compare with 50.

Since: 20 < 50 , Shift 50 right. Insert 20.

Array becomes: 20 50 10 40 30

Pass 2

Key element: 10

Compare with: 50 , 20

Shift both right and insert 10.

Array becomes: 10 20 50 40 30

Pass 3

Key element:40

Compare with 50. Shift 50 right. Insert 40.

Array becomes: 10 20 40 50 30

Pass 4

Key element: 30

Compare with: 50 and 40

Shift both right and Insert 30.

Final sorted array: 10 20 30 40 50

2. Program for Insertion Sort

```c
#include<stdio.h>

int main()
{
   int arr[5] = {50,20,10,40,30};

   int i,j,key;

   // Outer loop for passes
   for(i=1;i<5;i++)
   {
      // Store current element
      key = arr[i];

      // Start comparing from previous element
      j = i - 1;

      // Shift elements greater than key
      while(j >= 0 && arr[j] > key)
      {
         arr[j+1] = arr[j];

         j--;
      }

      // Insert key at correct position
      arr[j+1] = key;
   }

   // Print sorted array
   printf("Sorted Array:\n");

   for(i=0;i<5;i++)
   {
      printf("%d ", arr[i]);
   }

   return 0;
}
```

Output

Sorted Array: 10 20 30 40 50

Program Explanation

Initial Array : 50 20 10 40 30

Pass 1 → i = 1

Key: key = 20

J: j = 0

Compare: 50 > 20 → True

Shift 50: —> 50 50 10 40 30

Insert 20: 20 50 10 40 30

Pass 2 → i = 2

Key: key = 10

Compare with: 50 and 20

Shift elements:

20 20 50 40 30

20 50 50 40 30

Insert 10: 10 20 50 40 30

Pass 3 → i = 3

Key: key = 40

Compare: 50 > 40 —> True

Shift 50: 10 20 50 50 30

Insert 40: 10 20 40 50 30

Pass 4 → i = 4

Key: key = 30

Compare with: 50 and 40

Shift both:

10 20 40 50 50

10 20 40 40 50

Insert 30: 10 20 30 40 50

Time Complexity

Case	Complexity
Best Case	O(n)
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity : O(1)

No extra memory required.

4.Quick Sort

Quick Sort is a fast sorting algorithm based on the Divide and Conquer technique.

It works by:

1. Selecting one element called **pivot**
2. Placing pivot at correct position
3. Dividing array into two parts
4. Recursively sorting both parts

Why Called Divide and Conquer?

Quick sort:

- Divides large problem into smaller subproblems
- Solves smaller problems
- Combines result automatically

Working Principle

Suppose array: 50 20 10 40 30

Choose pivot: 30

Arrange elements: 10 20 | 30 | 50 40

Now:

- Left side contains smaller elements
- Right side contains larger elements

Again apply quick sort on both sides.

Final sorted array: 10 20 30 40 50

Important Terms

Term	Meaning
Pivot	Reference element
Partition	Dividing array around pivot
Recursive Calls	Sorting smaller subarrays

Algorithm

Step 1: Start

Step 2: Read

- Read the number of elements n.
- Read the array elements arr[].

Step 3: Call QuickSort

- Call QuickSort(arr, low, high) where:
    - low = 0
    - high = n - 1

Step 4: QuickSort Procedure

1. If low < high:
    - Find the partition position p.
    - Recursively sort the left subarray:
        - QuickSort(arr, low, p-1)
    - Recursively sort the right subarray:
        - QuickSort(arr, p+1, high)

Step 5: Stop

Partition Algorithm

1. Choose the first element as the pivot.
2. Set:
    - i = low + 1
    - j = high
3. Move i from left to right until an element greater than the pivot is found.
4. Move j from right to left until an element less than or equal to the pivot is found.
5. If i < j, swap arr[i] and arr[j].
6. Repeat until i >= j.
7. Swap the pivot element with arr[j].
8. Return j as the partition index.

Pseudocode

```
Main Quick Sort
```

```
QuickSort(arr, low, high)
{
   if(low < high)
   {
       p = Partition(arr, low, high)

       QuickSort(arr, low, p-1)
       QuickSort(arr, p+1, high)
   }
}

Partition Function

Partition(arr, low, high)
{
   pivot = arr[low]

   i = low + 1
   j = high

   while(i <= j)
   {
       while(i <= high && arr[i] <= pivot)
           i++

       while(arr[j] > pivot)
           j--

       if(i < j)
           swap(arr[i], arr[j])
   }

   swap(arr[low], arr[j])

   return j
}
```

Example

Initial array: 50 20 10 40 30

Pass 1

Pivot: 30

Partition: 20 10 | 30 | 50 40

Left Subarray : 20 10

Pivot: 10

Sorted: 10 20

Right Subarray : 50 40

Pivot: 40

Sorted: 40 50

Final Array : 10 20 30 40 50

C Program for Quick Sort

```
#include<stdio.h>

// Function to swap elements
void swap(int *a, int *b)
{
   int temp;

   temp = *a;
   *a = *b;
   *b = temp;
}

// Partition function
int partition(int arr[], int low, int high)
{
   // Select last element as pivot
   int pivot = arr[high];

   int i = low - 1;
```

```
    int j;

    // Compare elements with pivot
    for(j = low; j < high; j++)
    {
        // If element smaller than pivot
        if(arr[j] < pivot)
        {
            i++;

            swap(&arr[i], &arr[j]);
        }
    }

    // Place pivot at correct position
    swap(&arr[i+1], &arr[high]);

    return i + 1;
}

// Quick sort function
void quickSort(int arr[], int low, int high)
{
    if(low < high)
    {
        // Find partition index
        int pi = partition(arr, low, high);

        // Sort left subarray
        quickSort(arr, low, pi - 1);

        // Sort right subarray
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int arr[5] = {50,20,10,40,30};

    int i;

    quickSort(arr, 0, 4);
```

```
    printf("Sorted Array:\n");

    for(i=0;i<5;i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

Sorted Array: 10 20 30 40 50

Program Explanation

swap() Function

swap(&arr[i], &arr[j]);

Used to exchange elements.

partition() Function

Main function of quick sort.

Purpose:

- Select pivot
- Arrange elements
- Return pivot position

Pivot Selection

int pivot = arr[high];

Last element selected as pivot.

Comparison Loop

for(j = low; j < high; j++)

Checks all elements.

Condition

if(arr[j] < pivot)

Smaller elements moved left side.

Final Pivot Placement

swap(&arr[i+1], &arr[high]);

Places pivot at correct position.

Recursive Calls

quickSort(arr, low, pi - 1);

quickSort(arr, pi + 1, high);

Sorts left and right subarrays.

Time Complexity

Case	Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$

5.Merge Sort

Merge Sort is a sorting algorithm based on the Divide and Conquer technique.

It:

1. Divides array into smaller parts
2. Sorts each part recursively
3. Merges sorted parts together

Why Called Merge Sort?

Because:

- Array is divided
- Then smaller sorted arrays are merged

Working Principle

Suppose array: 50 20 10 40 30

Step 1: Divide Array

50 20 10 | 40 30

Again divide:

50 | 20 10

40 | 30

Again divide:

50

20 | 10

40

30

Step 2: Merge Smaller Arrays

Sort and merge: 20 10 -> 10 20

Merge: 50 + 10 20

Result: 10 20 50

Similarly: 40 30 -> 30 40

Final merge: 10 20 50 + 30 40

Final sorted array: 10 20 30 40 50

Algorithm

Step 1: Start

Step 2: Read

- Read the number of elements n.
- Read the array elements arr[].

Step 3: Call MergeSort

- Call MergeSort(arr, low, high)
    - low = 0
    - high = n - 1

Step 4: Divide the Array

1. If low < high:
    - Find the middle element:
        - mid = (low + high) / 2
    - Recursively sort the left half:
        - MergeSort(arr, low, mid)
    - Recursively sort the right half:
        - MergeSort(arr, mid+1, high)
    - Merge the two sorted halves.

Step 5: Stop

Pseudocode: Merge Sort

```
MergeSort(arr, low, high)
{
   if(low < high)
   {
       mid = (low + high) / 2
       MergeSort(arr, low, mid)
       MergeSort(arr, mid+1, high)

       Merge(arr, low, mid, high)
   }
}

Pseudocode: Merge Function
Merge(arr, low, mid, high)
{
   i = low
   j = mid + 1
   k = low
   Create temporary array temp[]
   while(i <= mid AND j <= high)
   {
       if(arr[i] <= arr[j])
       {
           temp[k] = arr[i]
           i++
       }
       else
       {
           temp[k] = arr[j]
           j++
       }
       k++
   }

   while(i <= mid)
   {
       temp[k] = arr[i]
       i++
       k++
   }

   while(j <= high)
```

```
    {
        temp[k] = arr[j]
        j++
        k++
    }

    for(k = low; k <= high; k++)
    {
        arr[k] = temp[k]
    }
}
```

Important Point

Single element array is already sorted.

Merge Sort Recursive Structure

Divide -> Divide -> Merge -> Merge

C Program for Merge Sort

```
#include<stdio.h>

// Function to merge arrays
void merge(int arr[], int low, int mid, int high)
{
  int temp[100];

  int i = low;
  int j = mid + 1;
  int k = low;

  // Compare and merge
  while(i <= mid && j <= high)
  {
     if(arr[i] < arr[j])
     {
        temp[k] = arr[i];

        i++;
```

```
        }
        else
        {
            temp[k] = arr[j];

            j++;
        }

        k++;
    }

    // Copy remaining left elements
    while(i <= mid)
    {
        temp[k] = arr[i];

        i++;
        k++;
    }

    // Copy remaining right elements
    while(j <= high)
    {
        temp[k] = arr[j];

        j++;
        k++;
    }

    // Copy sorted data back to original array
    for(i = low; i <= high; i++)
    {
        arr[i] = temp[i];
    }
}

// Merge sort function
void mergeSort(int arr[], int low, int high)
{
    int mid;

    if(low < high)
    {
```

```
        // Find middle index
        mid = (low + high) / 2;

        // Sort left half
        mergeSort(arr, low, mid);

        // Sort right half
        mergeSort(arr, mid + 1, high);

        // Merge sorted halves
        merge(arr, low, mid, high);
    }
}

int main()
{
    int arr[5] = {50,20,10,40,30};

    int i;

    mergeSort(arr, 0, 4);

    printf("Sorted Array:\n");

    for(i=0;i<5;i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

Sorted Array: 10 20 30 40 50

Time Complexity

Case	Complexity

Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)

Space Complexity : O(n)

Extra memory required for temporary array.

Sorting Comparison Table

Algorithm	Best Case	Worst Case
Bubble Sort	O(n)	O(n²)
Selection Sort	O(n²)	O(n²)
Insertion Sort	O(n)	O(n²)
Quick Sort	O(n log n)	O(n²)
Merge Sort	O(n log n)	O(n log n)

Two-Dimensional Array

A **Two-Dimensional Array** is an array that stores data in the form of **rows and columns**, similar to a table or matrix.

It is also called a **Matrix Array** because it resembles a mathematical matrix.

Syntax

datatype array[row][column];

Example:

int arr[2][3];

Memory Layout of 2D Array

Row	Column
arr[0][0]	arr[0][1]
arr[1][0]	arr[1][1]

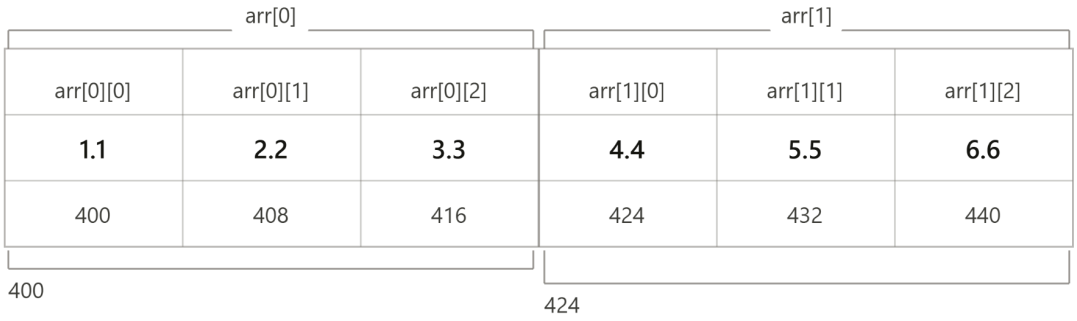


Figure 8.2 : 2-D Array Physical Layout

Each element is a double (8 bytes) — addresses increment by 8
arr[0] starts at 400 · arr[1] starts at 424 · arr[0][0] to arr[1][2] = 400 to 440

Declaration and Initialization of Two-Dimensional Arrays

A **2D array** is declared using two dimensions:

data_type array_name[row_size][column_size];

Example: int arr[3][4];

This creates an array with **3 rows** and **4 columns**.

1. Declaration Only (Valid)

Memory is allocated, but values are not initialized memory location holds the garbage values.

int arr[3][4];

Representation

Row 0 → arr[0][0] | arr[0][1] | arr[0][2] | arr[0][3]
Row 1 → arr[1][0] | arr[1][1] | arr[1][2] | arr[1][3]
Row 2 → arr[2][0] | arr[2][1] | arr[2][2] | arr[2][3]

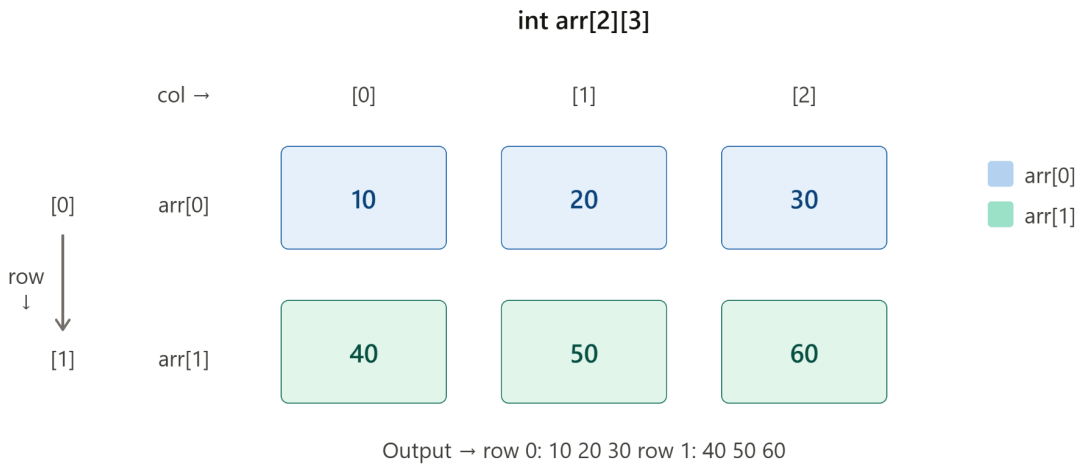
2. Complete Initialization (Valid)

Specify all rows and columns.

```
int arr[2][3] =
{
  {10, 20, 30},
  {40, 50, 60}
};
```

Output:

```
10 20 30
40 50 60
```



3. Flat (Row-wise) Initialization (Valid)

Elements are filled row by row automatically.

```
int arr[2][3] =
{
  10, 20, 30,
  40, 50, 60
};
```

Compiler interprets it as:

```
int arr[2][3] =
{
  {10,20,30},
  {40,50,60}
};
```

4. Partial Initialization (Valid)

Unspecified elements become zero.

```
int arr[2][3] =
{
  {10,20},
  {40}
};
```

Stored as:

```
10 20 0
40 0  0
```

5. Single Row Initialization (Valid)

```
int arr[2][3] =
{
  {10,20,30}
};
```

Stored as:

```
10 20 30
0  0  0
```

6. Omitting Row Size (Valid)

Compiler calculates the number of rows.

```
int arr[][3] =
{
  {10,20,30},
  {40,50,60}
};
```

Compiler determines:

```
int arr[2][3];
```

7. Omitting Both Dimensions (Invalid)

```
int arr[][];
```

Reason

Compiler cannot determine:

- Number of rows
- Number of columns

8. Omitting Column Size During Initialization (Invalid)

```
int arr[] =
{
  {10,20,30},
  {40,50,60}
};
```

Reason

This syntax is only for 1D arrays.

9. Omitting Column Size (Invalid)

```
int arr[2][];
```

Reason

In C, the column size must always be known.

10. Excess Initializers (Invalid)

```
int arr[2][3] =
{
  {10,20,30,40},
  {50,60,70}
};
```

Reason

Each row can contain only 3 elements.

11. More Rows Than Declared (Invalid)

```
int arr[2][3] =
{
  {10,20,30},
  {40,50,60},
  {70,80,90}
};
```

Reason

Array has only 2 rows.

12. Fewer Elements Than Size (Valid)

```
int arr[2][3] =
{
  {10},
  {20}
};
```

Stored as:

```
10 0 0
20 0 0
```

13. Zero Initialization (Valid)

```
int arr[3][4] = {0};
```

All elements become zero.

```
0 0 0 0
0 0 0 0
0 0 0 0
```

14. Character 2D Array (String Table) (Valid)

```
char names[3][10] =
{
  "Ram",
  "Sham",
  "Mohan"
};
```

15. Mixed Initialization Style (Valid)

```
int arr[2][3] =
{
  {10,20,30},
  40,50,60
};
```

Most modern C compilers accept this and interpret it as:

```
{
  {10,20,30},
  {40,50,60}
}
```

Summary Table

Declaration / Initialization	Valid / Invalid	Reason
int arr[3][4];	Valid	Normal declaration
int arr[2][3]={{1,2,3},{4,5,6}};	Valid	Complete initialization
int arr[2][3]={1,2,3,4,5,6};	Valid	Row-wise initialization
int arr[][3]={{1,2,3},{4,5,6}};	Valid	Row size inferred

int arr[2][3]={{1,2},{3}};	Valid	Partial initialization
int arr[3][4]={0};	Valid	All elements zero
int arr[][];	Invalid	Both dimensions missing
int arr[2][];	Invalid	Column size missing
int arr[]={{1,2},{3,4}};	Invalid	Wrong syntax
int arr[2][3]={{1,2,3,4},{5,6,7}};	Invalid	Too many columns
int arr[2][3]={{1,2,3},{4,5,6},{7,8,9}};	Invalid	Too many rows

Example Program

```c
#include <stdio.h>

int main()
{
    // 2D array with 2 rows and 3 columns
    int arr[2][3] =
    {
        {1, 2, 3},
        {4, 5, 6}
    };

    int i, j;

    // Traverse rows and columns
    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
        {
            printf("%d ", arr[i][j]);
        }

        // Move to next row
        printf("\n");
    }
```

```
    return 0;
}
```

Passing 2D Array to Function

A **2D array** can be passed to a function just like a 1D array. However, while passing a 2D array, the **number of columns must be specified** in the function parameter.

General Syntax

Declaration
int arr[3][4];

Function Prototype
void display(int a[][4], int rows);

OR

void display(int a[3][4]);

OR

void display(int (*a)[4], int rows);

Why Column Size is Mandatory?

A 2D array is stored in memory row-wise.

Example:

int arr[2][3] =
{
  {10,20,30},
  {40,50,60}
};

Memory Layout:

Address	Value
1000	10

1002	20
1004	30
1006	40
1008	50
1010	60

To access: arr[i][j]

Compiler calculates:

Address = Base + ((i * No_of_Columns) + j) * Size

Therefore, the compiler **must know the number of columns**.

Program 1: Display a 2D Array Using Function

```
#include<stdio.h>

void display(int a[][3], int rows);

int main()
{
   int arr[2][3] =
   {
      {10,20,30},
      {40,50,60}
   };

   display(arr,2);

   return 0;
}

void display(int a[][3], int rows)
{
   int i,j;

   for(i=0;i<rows;i++)
   {
      for(j=0;j<3;j++)
```

```
    {
        printf("%d ",a[i][j]);
    }
    printf("\n");
  }
}
```

Output

10 20 30
40 50 60

Program 2: Read and Display 2D Array Using Function

```
#include<stdio.h>

// Function prototypes
void readArray(int a[][3], int rows);
void displayArray(int a[][3], int rows);

int main()
{
    // Declare a 2D array with 2 rows and 3 columns
    int arr[2][3];

    // Read array elements from the user
    readArray(arr, 2);

    printf("\nArray Elements:\n");

    // Display array elements
    displayArray(arr, 2);

    return 0;
}

// Function to read elements into the 2D array
void readArray(int a[][3], int rows)
{
    int i, j;

    printf("Enter Elements:\n");
```

```
    // Traverse rows and columns to read elements
    for(i=0; i<rows; i++)
    {
        for(j=0; j<3; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

// Function to display elements of the 2D array
void displayArray(int a[][3], int rows)
{
    int i, j;

    // Traverse rows and columns to display elements
    for(i=0; i<rows; i++)
    {
        for(j=0; j<3; j++)
        {
            printf("%d ", a[i][j]);
        }

        // Move to the next line after each row
        printf("\n");
    }
}
```

Program 3: Find Sum of All Elements of a 2D Array

```
#include<stdio.h>

// Function prototype
int sumArray(int (*a)[3], int rows);

int main()
{
    // Declare and initialize a 2D array
```

```
    int arr[2][3] =
    {
        {10,20,30},
        {40,50,60}
    };

    // Call function to calculate and display sum
    printf("Sum = %d", sumArray(arr,2));

    return 0;
}

// Function to calculate sum of all array elements
int sumArray(int (*a)[3], int rows)
{
    int i,j,sum=0;

    // Traverse each row
    for(i=0;i<rows;i++)
    {
        // Traverse each column
        for(j=0;j<3;j++)
        {
            // Access array element using pointer notation
            sum = sum + *(*(a+i)+j);
        }
    }

    // Return the calculated sum
    return sum;
}
```

Output

Sum = 210

Different Ways to Pass a 2D Array

Method 1 (Most Common)

void fun(int a[][3])

{

}

Method 2
void fun(int a[2][3])
{
}

Method 3 (Using Pointer)
void fun(int (*a)[3])
{
}

Matrix Addition Program

```
#include<stdio.h>

int main()
{
    // Declare and initialize two 2D arrays
    int a[2][2] = {{1,2},{3,4}};
    int b[2][2] = {{5,6},{7,8}};

    // Array to store the sum of a and b
    int c[2][2];

    int i,j;

    // Add corresponding elements of both arrays
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    }

    // Display the resultant array
    for(i=0;i<2;i++)
    {
        for(j=0;j<2;j++)
        {
            printf("%d ", c[i][j]);
```

```
        }

        // Move to the next row
        printf("\n");
    }

    return 0;
}
```

Applications of Arrays in Embedded Systems

Arrays used in:

- ADC sample storage
- UART buffer
- Sensor buffer
- LCD display buffer
- Audio processing
- Image processing
- DMA transfer

PART 2: STRINGS

Introduction to String

A **String** is a collection (sequence) of characters terminated by a special character called the **NULL character (`'\0'`)**.

Definition

> A string is a group of characters stored in consecutive memory locations and terminated by the NULL character (`'\0'`).

Example

char str[] = "Hello";

Although it looks like only 5 characters are stored, C actually stores:

‘H’ ‘e’ ‘l’ ‘l’ ‘o’ ‘\0’

Memory representation:

Index	Value
str[0]	H
str[1]	e
str[2]	l
str[3]	l
str[4]	o
str[5]	'\0'

What is NULL Character ('\0')?

The NULL character is '\0' has ASCII value is 0. It is not the same as '0' which has ASCII value is 48.

Character	ASCII Value
'0'	48
'\0'	0

Why is '\0' Required?

The NULL character tells the compiler and string functions where the string ends.

Example: char str[] = "Hello";

When printf("%s", str); executes:

printf("%s", str);

The compiler prints:

H
e
l
l
o

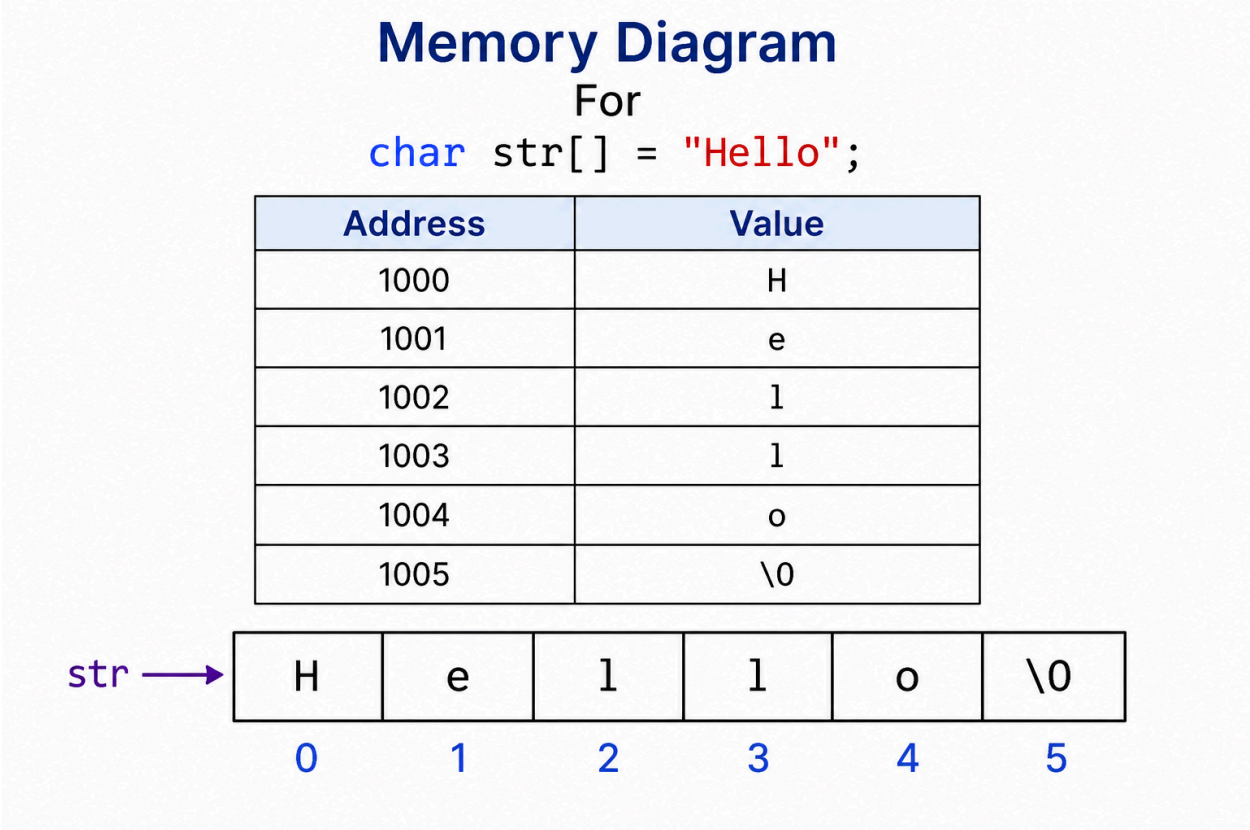
and stops when it finds:

'\0'

Memory Diagram

For

char str[] = "Hello";



Example Program

```c
#include<stdio.h>

int main()
{
   char str[] = "Hello";

   printf("%s", str);

   return 0;
}
```

Output
Hello

How printf("%s") Works

Internally: char str[] = "Hello";

is processed like:

```
int i = 0;

while(str[i] != '\0')
{
   printf("%c", str[i]);
   i++;
}
```

When str[i] becomes '\0', the loop stops.

String Initialization Methods

Method 1
char str[] = "Hello";

Compiler automatically adds '\0'.

Stored as: H e l l o \0

Method 2
char str[] = {'H','e','l','l','o','\0'};

Stored as: H e l l o \0

Method 3
char str[6] = {'H','e','l','l','o','\0'};

What Happens Without '\0'?

char str[] = {'H','e','l','l','o'};

Here no NULL character exists.

Now:

printf("%s", str);

may produce:

HelloABCDxyz...

or garbage output because printf() keeps reading memory until it accidentally finds a '\0'.

This is why every string in C **must end with a NULL character**.

Character Array vs String

Character Array
char arr[] = {'H','e','l','l','o'};

- No '\0'
- Not a proper string

String
char str[] = "Hello";

Equivalent to:

char str[] = {'H','e','l','l','o','\0'};

- Contains '\0'
- Proper string

Taking String Input from the user

There are several ways to take a string as input from the user.

1. Using scanf("%s", str)

Syntax

scanf("%s", str);

Program

```c
#include<stdio.h>
int main()
{
   char str[20];

   printf("Enter String: ");
   scanf("%s", str);

   printf("String = %s", str);

   return 0;
}
```

Input : Hello

Output : String = Hello

Limitation

If the user enters: Hello World

Output:

String = Hello

scanf("%s") stops reading when it encounters a space, tab, or newline.

2. Using Scanset (scanf("%[^\n]"))

Syntax

scanf("%[^\n]", str);

Program

```c
#include<stdio.h>

int main()
{
   char str[50];

   printf("Enter String: ");
   scanf("%[^\n]", str);

   printf("String = %s", str);

   return 0;
}
```

Input : Hello World

Output

String = Hello World

Working

%[^\n]

means:

- Read all characters
- Until newline (\n) is encountered

3. Using gets() (Obsolete / Unsafe)

Syntax : gets(str);

Program

```c
#include<stdio.h>

int main()
{
   char str[50];

   printf("Enter String: ");
```

```
    gets(str);

    printf("String = %s", str);

    return 0;
}
```

Input : Hello World

Output : String = Hello World

Drawback

- No boundary checking
- Can cause buffer overflow
- Removed from modern C standards

Avoid using gets() in new programs.

4. Using fgets() (Recommended)

Syntax

fgets(str, sizeof(str), stdin);

Program

```
#include<stdio.h>

int main()
{
    char str[50];

    printf("Enter String: ");
    fgets(str, sizeof(str), stdin);

    printf("String = %s", str);

    return 0;
}
```

Input

Hello World

Output

String = Hello World

Advantages

- Reads spaces
- Prevents buffer overflow
- Recommended in modern C

Note

fgets() stores the newline character if space is available.

Example:

Input: Hello

Stored as:

H e l l o \n \0

5. Using Character-by-Character Input (getchar())

Program

```
#include<stdio.h>

int main()
{
   char str[100];
   int i = 0;

   printf("Enter String: ");

   while((str[i] = getchar()) != '\n')
   {
      i++;
   }

   str[i] = '\0';
```

```
    printf("String = %s", str);

    return 0;
}
```

Input : Hello World

Output : String = Hello World

Working

- Reads one character at a time
- Stops when Enter (\n) is pressed
- Adds '\0' at the end

6. Using Pointer and getchar()

Program

```
#include<stdio.h>

int main()
{
    char str[100];
    int i = 0;

    printf("Enter String: ");

    while((str[i] = getchar()) != '\n')
    {
        i++;
    }

    str[i] = '\0';

    printf("String = %s", str);

    return 0;
}
```

Comparison Table

Method	Reads Spaces	Safe	Recommended
scanf("%s")	No	Yes	For single word
scanf("%[^\n]")	Yes	Limited	Sometimes
gets()	Yes	No	Never
fgets()	Yes	Yes	Best
getchar() Loop	Yes	Yes	Good for learning
Pointer + getchar()	Yes	Yes	Good for pointers

Example Program

```
#include<stdio.h>

int main()
{
   char str[50];

   printf(\"Enter String: \");

   fgets(str,sizeof(str),stdin);

   printf(\"%s\", str);

   return 0;
}
```

String Output

In C, there are several methods to display output on the console.

1. printf()

The most commonly used function.

Syntax:

printf("format string", variables);

Example:

```
#include<stdio.h>

int main()
{
   int a = 10;
   printf("Value of a = %d", a);
   return 0;
}
```

Output:

Value of a = 10

2. puts()

Displays a complete string and automatically moves the cursor to the next line.

Syntax:

puts(string);

Example

```
#include<stdio.h>
```

```c
int main()
{
   puts("Hello World");
   return 0;
}
```

Output: Hello World

Equivalent to: printf("Hello World\n");

3. putchar()

Displays a single character.

Syntax:

putchar(character);

Example:

```c
#include<stdio.h>

int main()
{
   putchar('A');
   return 0;
}
```

Output:

A

4. fputs()

Displays a string to a specified output stream.

Syntax:

fputs(string, stdout);

Example:

```c
#include<stdio.h>

int main()
{
   fputs("Hello World", stdout);
   return 0;
}
```

Output:

Hello World

5. fprintf()

Works like printf() but allows output to any stream.

Syntax:

fprintf(stdout, "Message");

Example:

```c
#include<stdio.h>

int main()
{
   int x = 100;

   fprintf(stdout, "Value = %d", x);

   return 0;
}
```

Output:

Value = 100

Equivalent to:

printf("Value = %d", x);

Displaying Character by Character Using Loop

Example:

```c
#include<stdio.h>

int main()
{
   char str[] = "Hello";

   int i;
   for(i = 0; str[i] != '\0'; i++)
   {
      putchar(str[i]);
   }

   return 0;
}
```

Output:

Hello

Using Escape Sequences

printf("Hello\n");
printf("World");

Output:

Hello
World

Common escape sequences:

Escape Sequence	Meaning
\n	New Line
\t	Horizontal Tab
\\	Backslash
\"	Double Quote

\b	Backspace
\r	Carriage Return

Summary

Function	Purpose
printf()	Display formatted output
puts()	Display string with newline
putchar()	Display single character
fprintf()	Display formatted output to a stream
fputs()	Display string to a stream
putchar() in loop	Display string character-by-character

String.h Library

Header file: #include<string.h>

Contains predefined string functions.

Common String Functions

1. strlen()
2. strcpy()
3. strcat()
4. strcmp()
5. strchr()
6. strstr()

1) strlen()

`strlen()` is a **string handling function** used to find the **length of a string**.

Prototype

size_t strlen(const char *str);

Syntax

strlen(string);

Example

```
#include<stdio.h>
#include<string.h>

int main()
{
   char str[] = \"Embedded\";

   int len;

   len = strlen(str);

   printf(\"Length = %d\", len);

   return 0;
}
```

2) strcpy()

`strcpy()` is a **string handling function** used to **copy one string into another string**.

It is declared in the **`<string.h>`** header file.

Prototype

char *strcpy(char *dest, const char *src);

Parameters

- `dest` → Destination string where the data is copied.
- `src` → Source string that is copied.

What does it return?

`strcpy()` returns a **pointer to the destination string (`dest`)**.

Syntax
strcpy(destination, source);

Example

```
#include<stdio.h>
#include<string.h>

int main()
{
   char src[] = \"Hello\";

   char dest[20];

   strcpy(dest, src);

   printf(\"%s\", dest);

   return 0;
}
```

3)strcat()

`strcat()` is a **string handling function** used to **concatenate (join) two strings**.

It is declared in the **`<string.h>`** header file.

Prototype
char *strcat(char *dest, const char *src);

Parameters

- `dest` → Destination string. The source string is appended to it.
- `src` → Source string that is added at the end of `dest`.

What does it return?

`strcat()` returns a **pointer to the destination string (`dest`)**.

Syntax
strcat(destination, source);

Example
```
#include<stdio.h>
#include<string.h>

int main()
{
   char str1[20] = \"Hello \";

   char str2[] = \"World\";

   strcat(str1,str2);

   printf(\"%s\", str1);

   return 0;
}
```

4)strcmp()

`strcmp()` is a **string handling function** used to **compare two strings**.

It is declared in the **`<string.h>`** header file.

Prototype
int strcmp(const char *str1, const char *str2);

What does it return?

`strcmp()` compares the strings **character by character** and returns an integer:

Return value	Meaning

`0`	Both strings are equal
`< 0`	`str1` is smaller than `str2`
`> 0`	`str1` is greater than `str2`

Syntax

strcmp(str1,str2);

Return Values

Return	Meaning
0	Equal
<0	str1 smaller
>0	str1 greater

Example

```
#include<stdio.h>
#include<string.h>

int main()
{
   char str1[] = \"ABC\";
   char str2[] = \"ABC\";

   if(strcmp(str1,str2)==0)
   {
      printf(\"Equal\");
   }

   return 0;
}
```

5)strchr()

`strchr()` is a **string handling function** used to **find the first occurrence of a character in a string**.

It is declared in the **`<string.h>`** header file.

Prototype
char *strchr(const char *str, int ch);

Parameters

- `str` → String in which the character is searched.
- `ch` → Character to search for.

What does it return?

- If the character is **found**, `strchr()` returns a **pointer to the first occurrence** of that character.
- If the character is **not found**, it returns `NULL`.

Syntax
strchr(string, character);

Example

```
#include<stdio.h>
#include<string.h>

int main()
{
    char str[] = "Hello World";
    char *ptr;

    ptr = strchr(str, 'W');

    if(ptr != NULL)
    {
        printf("Character found");
    }
    else
    {
        printf("Character not found");
```

```
    }

    return 0;
}
```

6)strstr()

Finds substring.

Syntax

strstr(main_string, sub_string);

Complete Program Using String Functions

```
#include<stdio.h>
#include<string.h>

int main()
{
  char str1[50] = \"Embedded\";

  char str2[50] = \" Systems\";

  char copy[50];

  int len;

  // Find length
  len = strlen(str1);

  printf(\"Length = %d\\n\", len);

  // Copy string
  strcpy(copy, str1);

  printf(\"Copied = %s\\n\", copy);

  // Concatenate strings
  strcat(str1, str2);
```

```
  printf(\"Concatenated = %s\\n\", str1);

  // Compare strings
  if(strcmp(copy,\"Embedded\") == 0)
  {
     printf(\"Strings Equal\\n\");
  }

  return 0;
}
```

String Functions Using Pointer

1. strlen() Using Pointer

Algorithm

1. Initialize counter to 0
2. Traverse string using pointer
3. Increment counter until '\0'
4. Return count

Program

```
#include<stdio.h>
#include<string.h>

int main()
{
  char str1[50] = \"Embedded\";

  char str2[50] = \" Systems\";

  char copy[50];
```

```
    int len;

    // Find length
    len = strlen(str1);

    printf(\"Length = %d\\n\", len);

    // Copy string
    strcpy(copy, str1);

    printf(\"Copied = %s\\n\", copy);

    // Concatenate strings
    strcat(str1, str2);

    printf(\"Concatenated = %s\\n\", str1);

    // Compare strings
    if(strcmp(copy,\"Embedded\") == 0)
    {
        printf(\"Strings Equal\\n\");
    }

    return 0;
}
```

Output

Length = 8

Explanation

while(*ptr != '\0')

Checks each character.

ptr++;

Moves pointer to next character.

2. strcpy() Using Pointer

Copies source string into destination string.

Algorithm

1. Read character from source
2. Copy to destination
3. Move both pointers
4. Repeat until NULL copied

Program

```c
#include<stdio.h>

// Function to copy string
void my_strcpy(char *dest, char *src)
{
   // Copy characters
   while(*src != '\0')
   {
      *dest = *src;

      dest++;
      src++;
   }

   // Add NULL character
   *dest = '\0';
}

int main()
{
   char src[] = "HELLO";

   char dest[20];

   my_strcpy(dest, src);

   printf("Copied String = %s", dest);

   return 0;
}
```

Output

Copied String = HELLO

3. strcmp() Using Pointer

Compares two strings.

Return Values

Return Value	Meaning
0	Strings equal
<0	First string smaller
>0	First string greater

Algorithm

1. Compare characters one by one
2. If mismatch found return difference
3. If NULL reached return 0

Program

```
#include<stdio.h>

// Function to compare strings
int my_strcmp(char *str1, char *str2)
{
  while(*str1 == *str2)
  {
     // If NULL reached
     if(*str1 == '\0')
     {
        return 0;
     }

     str1++;
     str2++;
  }
```

```
    return *str1 - *str2;
}

int main()
{
    char str1[] = "ABC";

    char str2[] = "ABC";

    int result;

    result = my_strcmp(str1, str2);

    if(result == 0)
    {
        printf("Strings are Equal");
    }
    else
    {
        printf("Strings are Not Equal");
    }

    return 0;
}
```

4. strcat() Using Pointer

Definition

Concatenates two strings. Adds source string at end of destination string.

Algorithm

1. Move destination pointer to NULL
2. Copy source string
3. Add NULL at end

Program

```
#include<stdio.h>
```

```c
// Function to concatenate strings
void my_strcat(char *dest, char *src)
{
   // Move to end of destination string
   while(*dest != '\0')
   {
      dest++;
   }

   // Copy source string
   while(*src != '\0')
   {
      *dest = *src;

      dest++;
      src++;
   }

   // Add NULL character
   *dest = '\0';
}

int main()
{
   char str1[30] = "Embedded ";

   char str2[] = "Systems";

   my_strcat(str1, str2);

   printf("Concatenated String = %s", str1);

   return 0;
}
```

5. strchr() Using Pointer

Definition

Finds first occurrence of character in string.

Program

```c
#include<stdio.h>

// Function to search character
char* my_strchr(char *str, char ch)
{
   while(*str != '\0')
   {
      if(*str == ch)
      {
         return str;
      }

      str++;
   }

   return NULL;
}

int main()
{
   char str[] = "Embedded";

   char *ptr;

   ptr = my_strchr(str, 'd');

   if(ptr != NULL)
   {
      printf("Character Found");
   }
   else
   {
      printf("Character Not Found");
   }

   return 0;
}
```

Complete Program of All Functions

```c
#include<stdio.h>

// strlen()
int my_strlen(char *ptr)
{
   int count = 0;

   while(*ptr != '\0')
   {
      count++;

      ptr++;
   }

   return count;
}

// strcpy()
void my_strcpy(char *dest, char *src)
{
   while(*src != '\0')
   {
      *dest = *src;

      dest++;
      src++;
   }

   *dest = '\0';
}

// strcat()
void my_strcat(char *dest, char *src)
{
   while(*dest != '\0')
   {
      dest++;
   }

   while(*src != '\0')
   {
      *dest = *src;
```

```
        dest++;
        src++;
    }

    *dest = '\0';
}

// strcmp()
int my_strcmp(char *str1, char *str2)
{
    while(*str1 == *str2)
    {
        if(*str1 == '\0')
        {
            return 0;
        }

        str1++;
        str2++;
    }

    return *str1 - *str2;
}

int main()
{
    char str1[50] = "Embedded";

    char str2[] = " Systems";

    char copy[50];

    int len;

    // strlen
    len = my_strlen(str1);

    printf("Length = %d\n", len);

    // strcpy
    my_strcpy(copy, str1);

    printf("Copied String = %s\n", copy);
```

```
    // strcat
    my_strcat(str1, str2);

    printf("Concatenated String = %s\n", str1);

    // strcmp
    if(my_strcmp(copy, "Embedded") == 0)
    {
        printf("Strings are Equal\n");
    }

    return 0;
}
```

Output

Length = 8
Copied String = Embedded
Concatenated String = Embedded Systems
Strings are Equal

Why Pointer-Based String Functions Important?

Pointers are faster because:

- Direct memory access
- Less indexing overhead
- Efficient traversal

Embedded System Applications

Used in:

- UART communication
- AT command parsing
- GPS data parsing
- LCD display systems
- Communication protocols

Important Embedded System Applications

Strings used in:

- UART communication
- GPS parsing
- AT commands
- LCD display
- Menu systems
- Communication protocols

Interview Questions

1. Difference between array and pointer?
2. Why array index starts from 0?
3. Explain contiguous memory allocation.
4. Difference between linear and binary search?
5. Difference between bubble and insertion sort?
6. Difference between character array and string?
7. Why NULL character important?
8. Difference between gets() and fgets()?
9. Difference between strcpy() and memcpy()?
10. Explain strlen() internally.

MCQs

Q1.

int arr[5];

Last valid index?

A) 5
B) 4
C) 0
D) 1

Answer: B

Q2.

printf(\"%d\", 2[arr]);

Meaning?

A) Invalid
B) arr[2]
C) Address
D) Error

Answer: B

Q3.

Which searching algorithm requires sorted array?

A) Linear Search
B) Binary Search
C) Bubble Sort
D) Insertion Sort

Answer: B

Q4.

Which string function returns length?

A) strcpy()
B) strcat()
C) strlen()
D) strcmp()

Answer: C

Q5.

Which character terminates string?

A) EOF
B) \n
C) \0
D) NULL pointer

Answer: C